

Pretrained models using VGG-19 - Lab 4

Lab Overview

- Introduce the VGG-19 convolutional neural network architecture and its role in image classification.
- Explain how deep CNNs process images through convolution, activation, pooling, flattening, and fully connected layers.
- Study the detailed structure of VGG-19, including its convolutional blocks and final classification layers.
- Implement the VGG-19 model in Jupyter Notebook using TensorFlow/Keras.
- Understand the purpose of each code block and the function of each line in the notebook.
- Apply essential preprocessing steps such as image resizing, normalization, and dataset loading.
- Train the model on an image dataset and monitor its learning process.
- Evaluate model performance using accuracy/loss curves, classification report, confusion matrix, and prediction results.
- Develop both theoretical understanding and practical experience in using VGG-19 for computer vision tasks.

Contents

Lab Overview.....	1
1. Introduction.....	3
2. VGG-19 Architecture in Detail	3
2.1 High-level structure.....	3
2.2 Layer-by-layer organization.....	4
2.3 Why VGG-19 uses many 3×3 convolutions	4
2.4 Design principles highlighted in the article	4
2.5 Strengths and limitations	5
3. Code walkthrough.....	5
3.1 Code Cell 1	5
3.2 Code Cell 3	6
3.3 Code Cell 5	6
3.4 Code Cell 7	7
3.5 Code Cell 9.....	8
3.6 Code Cell 11	9
3.7 Code Cell 13	9
3.8 Code Cell 15	9
3.9 Code Cell 17	10
3.10 Code Cell 19.....	11

1. Introduction

VGG-19 is one of the landmark convolutional neural network architectures in deep learning. It was introduced by the Visual Geometry Group (VGG) at the University of Oxford and became well known for showing that simply increasing network depth in a disciplined way can produce strong image-classification performance. The model is called VGG-19 because it contains 19 weight-bearing layers: 16 convolutional layers and 3 fully connected layers.

This document has two goals. First, it explains the VGG-19 architecture in a clear academic style following the structure of the referenced GeeksforGeeks article. Second, it converts the supplied Jupyter notebook into a guided Word handout, where each code block is reproduced and each line is explained in detail.

2. VGG-19 Architecture in Detail

According to the GeeksforGeeks article, the core idea of VGG-19 is remarkably uniform: every convolution uses a 3×3 kernel, stride 1, and padding 1; ReLU follows each convolution; and max-pooling layers with a 2×2 window and stride 2 are used to progressively reduce spatial resolution. At the end, three fully connected layers perform classification, and a softmax layer converts the final outputs into class probabilities.

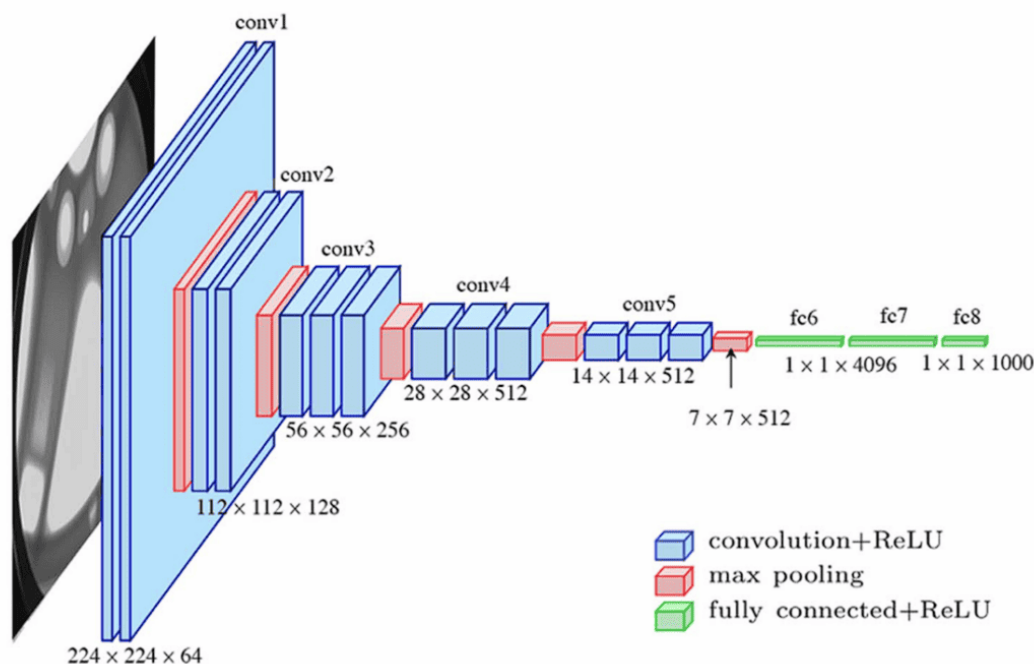


Figure 1. VGG-19 architecture diagram used in the lab.

2.1 High-level structure

The architecture is arranged as five convolutional blocks followed by a classifier head. As depth increases, the number of channels grows from 64 to 128, then 256, then 512, and finally 512 again.

Meanwhile, max pooling halves the spatial size after each block: $224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$. This means the model gradually trades spatial detail for richer semantic features.

2.2 Layer-by-layer organization

Block	Layers	Filters	Output Size	Purpose
Block 1	Conv1_1, Conv1_2, MaxPool	64	$224 \times 224 \rightarrow 112 \times 112$	Learns simple edges, corners, and low-level texture patterns.
Block 2	Conv2_1, Conv2_2, MaxPool	128	$112 \times 112 \rightarrow 56 \times 56$	Builds slightly more complex shapes and local motifs.
Block 3	Conv3_1 to Conv3_4, MaxPool	256	$56 \times 56 \rightarrow 28 \times 28$	Captures richer visual compositions and repeated structures.
Block 4	Conv4_1 to Conv4_4, MaxPool	512	$28 \times 28 \rightarrow 14 \times 14$	Learns higher-level abstractions such as object parts.
Block 5	Conv5_1 to Conv5_4, MaxPool	512	$14 \times 14 \rightarrow 7 \times 7$	Produces deep semantic feature maps used for final classification.
Classifier	FC1, FC2, FC3 + Softmax	4096, 4096, 1000	Class scores	Maps extracted features to class probabilities.

2.3 Why VGG-19 uses many 3×3 convolutions

Using repeated 3×3 convolutions is one of the defining design choices of VGG-style networks. Stacking multiple small kernels increases the effective receptive field while introducing more nonlinear activations than a single large kernel. For example, two 3×3 convolutions approximate the coverage of a 5×5 region, while three 3×3 convolutions approximate a 7×7 region. This gives the model expressive power without abandoning architectural simplicity.

2.4 Design principles highlighted in the article

- **Uniformity:** the same 3×3 convolution rule is used throughout the network, making the architecture easy to understand and reproduce.
- **Depth:** a deeper network can learn progressively more abstract features, from edges to textures to object parts and category-level concepts.

- ReLU activations: nonlinear activation after each convolution helps the network model complex decision boundaries.
- Max pooling: spatial dimensions are reduced in a controlled way, lowering computational load and increasing translational robustness.
- Strong transfer-learning behavior: pretrained VGG-19 weights are commonly reused as a generic image-feature extractor.

2.5 Strengths and limitations

VGG-19 is historically important because of its simplicity, strong feature extraction, and heavy use in transfer learning. However, it is also computationally expensive compared with later architectures such as ResNet, because it contains many parameters and large fully connected layers. This is why modern lab exercises often use VGG-19 as a pretrained backbone rather than training the full network from scratch.

3. Code walkthrough

The following subsections reproduce the main code blocks from the notebook and explain the purpose of each block in short paragraphs. Where the notebook includes a repeated step, such as importing again or compiling the model more than once, it is still documented so the report remains faithful to the original workflow.

3.1 Code Cell 1

Notebook context: 2) Import libraries. This cell gathers all required imports in one place.

```
import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
import kagglehub

from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
from tensorflow.keras import layers, models
from tensorflow.keras.applications import VGG19
from tensorflow.keras.applications.vgg19 import preprocess_input
from tensorflow.keras.preprocessing import image

print("TensorFlow version:", tf.__version__)
```

This code cell prepares the notebook environment by importing all libraries needed for the lab. It includes utilities for folder handling (os), numerical computation (numpy), plotting (matplotlib), deep learning (tensorflow), and dataset access (kagglehub). It also imports evaluation tools from scikit-learn and the pretrained VGG19 model with its required preprocessing function from Keras. Finally, it prints the installed TensorFlow version, which is useful for checking compatibility and reproducibility.

3.2 Code Cell 3

Notebook context: 3) Download and locate the dataset. We download the dataset once, then point to its data folder.

```
# Download latest version
dataset_path = r"C:\Users\Lenovo\.cache\kagglehub\datasets\kritikseth\fruit-and-vegetable-
image-recognition\versions\8\train"
print("Dataset path:", dataset_path)
print("Classes found:", os.listdir(dataset_path))
```

This code cell defines the local path of the dataset folder that will be used for training and validation. It then prints that path so the user can confirm that the notebook is pointing to the correct location. After that, it lists the subfolders inside the dataset directory, which helps verify that the classes are arranged properly because each subfolder represents one image category.

3.3 Code Cell 5

Notebook context: 4) Load the training and validation sets. We split the dataset into 80% training and 20% validation.

```
IMG_SIZE = (224, 224) # VGG19 standard input size
BATCH_SIZE = 32
SEED = 123

train_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical"
)
```

```
val_data = tf.keras.utils.image_dataset_from_directory(
    dataset_path,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical"
)

class_names = train_data.class_names
num_classes = len(class_names)

print("Class names:", class_names)
print("Number of classes:", num_classes)
```

This code cell loads the image dataset directly from the folder structure and splits it into training and validation subsets. The images are resized to 224 x 224 pixels, which is the standard input size expected by VGG19, and they are grouped into batches of 32 for efficient training. The split is made reproducible by setting a fixed random seed. After loading the data, the code extracts the class names and counts the number of classes so the model's output layer can later be configured correctly.

3.4 Code Cell 7

Notebook context: 5) Preprocess data for VGG19. VGG19 expects images to be preprocessed using `preprocess_input(...)` instead of simple `1/255` scaling. We also cache and prefetch the datasets to improve performance.

```
AUTOTUNE = tf.data.AUTOTUNE

train_data = train_data.map(lambda x, y: (preprocess_input(tf.cast(x, tf.float32)), y))
val_data = val_data.map(lambda x, y: (preprocess_input(tf.cast(x, tf.float32)), y))

train_data = train_data.cache().prefetch(AUTOTUNE)
val_data = val_data.cache().prefetch(AUTOTUNE)
```

This code cell applies to the correct preprocessing required by the pretrained VGG19 network. Instead of simple normalization, the images are converted to floating-point format and passed through `preprocess_input`, which transforms them into the format expected by ImageNet-pretrained

VGG19 weights. The cell also improves efficiency by caching the processed datasets in memory and prefetching future batches, which helps reduce delays during training and validation.

3.5 Code Cell 9

Notebook context: 6) Build the VGG19 model. We use VGG19 pretrained on ImageNet as a feature extractor, then add a small classification head for our dataset.

```
from tensorflow.keras.applications import VGG19
from tensorflow.keras import layers, models

base_model = VGG19(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(num_classes, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

This code cell constructs the classification model using transfer learning. First, the pretrained VGG19 backbone is loaded with ImageNet weights, while its original top classification layers are removed so it can be adapted to the current dataset. The backbone is frozen, so its learned visual features remain unchanged during the initial training stage. A small custom classifier is then added on top, consisting of global average pooling, a dense hidden layer, dropout for regularization, and a final softmax output layer sized to match the number of dataset classes. The model is then compiled using Adam, categorical cross-entropy, and accuracy.

3.6 Code Cell 11

Notebook context: 7) Compile the model. We use Adam optimizer, categorical cross-entropy loss, and accuracy as the main metric.

```
model.compile(  
    optimizer="adam",  
    loss="categorical_crossentropy",  
    metrics=["accuracy"]  
)  
model.summary()
```

This code cell compiles the model again and then prints a summary of the network architecture. The compile step defines how the model will learn by choosing the optimizer, the loss function, and the metric used for monitoring performance. After that, `model.summary()` displays the layers of the model, their output shapes, and the number of trainable and non-trainable parameters, which is useful for checking that the architecture has been built correctly.

3.7 Code Cell 13

Notebook context: 8) Train the model.

```
history = model.fit(  
    train_data,  
    validation_data=val_data,  
    epochs=10  
)
```

This code cell starts the model training process. The network is trained on the training dataset for 10 epochs, while the validation dataset is used at the end of each epoch to monitor how well the model generalizes unseen data. The results are stored in the historical object, which contains the recorded training and validation accuracy and loss values for later visualization.

3.8 Code Cell 15

Notebook context: 9) Plot training and validation performance.

```
plt.figure()  
plt.plot(history.history["accuracy"], label="Train Accuracy")
```

```
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("VGG19 Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

plt.figure()
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("VGG19 Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

This code cell visualizes the model's performance during training. The first plot shows how training accuracy and validation accuracy change across epochs, while the second plot shows the corresponding training and validation loss. These graphs help assess whether the model is improving steadily and whether there are signs of overfitting, such as training performance improving while validation performance stops improving or becoming worse.

3.9 Code Cell 17

Notebook context: 10) Classification report and confusion matrix.

```
y_true = []
y_pred = []

for images, labels in val_data:
    predictions = model.predict(images, verbose=0)
    y_true.extend(np.argmax(labels.numpy(), axis=1))
    y_pred.extend(np.argmax(predictions, axis=1))

y_true = np.array(y_true)
y_pred = np.array(y_pred)

print(classification_report(y_true, y_pred, target_names=class_names))

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
```

```
disp.plot(cmap="Blues")  
plt.title("Confusion Matrix")  
plt.show()
```

This code cell performs detailed evaluation on the validation dataset. It loops over all validation batches, obtains the model's predictions, and converts both the true one-hot labels and predicted probability vectors into class indices. These are then used to generate a classification report containing precision, recall, and F1-score for each class. The code also computes and displays a confusion matrix, which visually shows how often each class is correctly classified and which classes are being confused with one another.

3.10 Code Cell 19

Notebook context: 11) Predict a new image. Update `img_path` to test any image you want.

```
img_path = r"C:\Users\Lenovo\.cache\kagglehub\datasets\kritikseth\fruit-and-vegetable-  
image-recognition\versions\8\test\cabbage\Image_1.jpg"  
  
img = image.load_img(img_path, target_size=IMG_SIZE)  
img_array = image.img_to_array(img)  
img_array = np.expand_dims(img_array, axis=0)  
img_array = preprocess_input(img_array.astype(np.float32))  
  
prediction = model.predict(img_array, verbose=0)[0]  
predicted_index = np.argmax(prediction)  
  
print("Predicted class:", class_names[predicted_index])  
print("Class probabilities:")  
for name, prob in zip(class_names, prediction):  
    print(f"{name}: {prob:.4f}")
```

This code demonstrates how the trained model can be used to classify a single new image. The selected test image is loaded from disk, resized to the VGG19 input size, converted into an array, and reshaped to include a batch dimension. The same preprocessing used during training is then applied so the image matches the format expected by the model. Finally, the model predicts the class probabilities, identifies the class with the highest probability, and prints both the predicted label and the full probability distribution across all classes.